Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. A. Morzenti)

Written exam - 11 February 2014 - Part I: Theory

WARNING - THE SYNTAX ANALYSIS EXERCISES MUST BE SOLVED BY THE ELR / ELL THEORY

NAME:

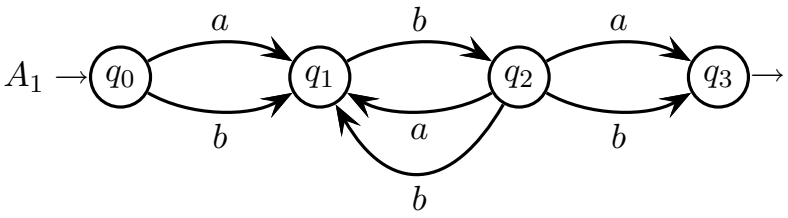
MATRICOLA: SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
  - I (80%) Theory:
    1. regular expressions and finite automata
    2. free grammars and pushdown automata
    3. syntax analysis and parsing
    4. translation and semantic analysis
  - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the finite automaton A_1 shown below:



and the following regular expression e_2:

$$e_2 = (\, b\; b\,)^*\; a$$

Answer the following questions, and explain the obtained results:

(a) By means of the *BMC* method (node elimination), write a regular expression e_1 such that the language equality $L\,(e_1) = L\,(A_1)$ holds. If possible, modify the regular expression e_1 to make it more compact.

(b) By means of the *BS* method (Berry-Sethi), build a deterministic finite automaton A_2 that recognizes language $L\,(e_2)$.

(c) By means of the automaton product construction (cartesian product), build a finite automaton A_3 that recognizes the intersection language $L\,(A_1) \cap L\,(A_2)$.

(d) If necessary, minimize the state number of the automaton A_3 found before, by means of a systematic method.

---

2 Free Grammars and Pushdown Automata 20%

1. Consider the following language L, over the alphabet $\{ a, b, c \}$:

$$L = \{ a^p b^q c^{p+q} c^* \mid \quad p, q \geq 0 \}$$

Answer the following questions:

(a) Write a grammar G that generates language L. The grammar must be not ambiguous and must be in the not extended form (BNF).

(b) (optional) Consider a new language L', over the same alphabet as language L, defined in this way:

$$L' = (\neg L) \cap (a^+ b^+ c^+)$$

that is, defined as the intersection between the complement of L and a regular expression. For such a new language L', write a grammar G' of any type, but not in the extended form (BNF).

---

2. One wishes one formalized a fragment of a hypothetical programming language, by considering only the logical expressions with the characteristics listed below:

   - The logical expressions contain only the operators "**not**", i.e., logical negation, and "**impl**", i.e., logical implication.
   - Negation has a priority higher than implication, i.e., negation takes precedence over implication, so that an expression like "**not** α **impl** β" must be interpreted as "(**not** α) **impl** β".
   - Implication is right-associative, thus an expression like "α **impl** β **impl** γ" must be interpreted as "α **impl** (β **impl** γ)".
   - The logical subexpressions may be contained in parentheses (round brackets), and this is the way to change the predefined precedence order between the operators or to change their associativity, like for instance in the expression "(α **impl** β) **impl** γ".
   - The elementary logical expressions are relational (equality) expressions of the form "$\alpha = \beta$", where the members α and β are numerical expressions.
   - The numerical expressions consist of the addition of an arbitrary number of terms "$\alpha_1 + \alpha_2 + \ldots + \alpha_k$" (with $k \geq 1$). A term is either the terminal **n** (which represents a generic number) or a subtraction of the form "$\mathbf{n} - \mathbf{n} - \ldots - \mathbf{n}$", with one minuend and one or more subtrahends.
   - Subtraction is left-associative, thus an expression like "$\mathbf{n} - \mathbf{n} - \mathbf{n}$" must be interpreted as "$(\mathbf{n} - \mathbf{n}) - \mathbf{n}$"; instead, the addition operation does not have any specific association order; therefore in the case of addition the grammar will have to be defined in such a way that in the syntax tree all the terms of an addition are brother nodes, i.e., they are all child nodes of one parent node.

   Notice that the numerical subexpressions *may not* contain parentheses.

   Here are a few sample phrases of this language:

   $\mathbf{n} = \mathbf{n} - \mathbf{n} + \mathbf{n} - \mathbf{n} - \mathbf{n} + \mathbf{n}$

   not $\mathbf{n} = \mathbf{n} - \mathbf{n}$

   not not $\mathbf{n} =$ **n impl not** $\mathbf{n} =$ **n**

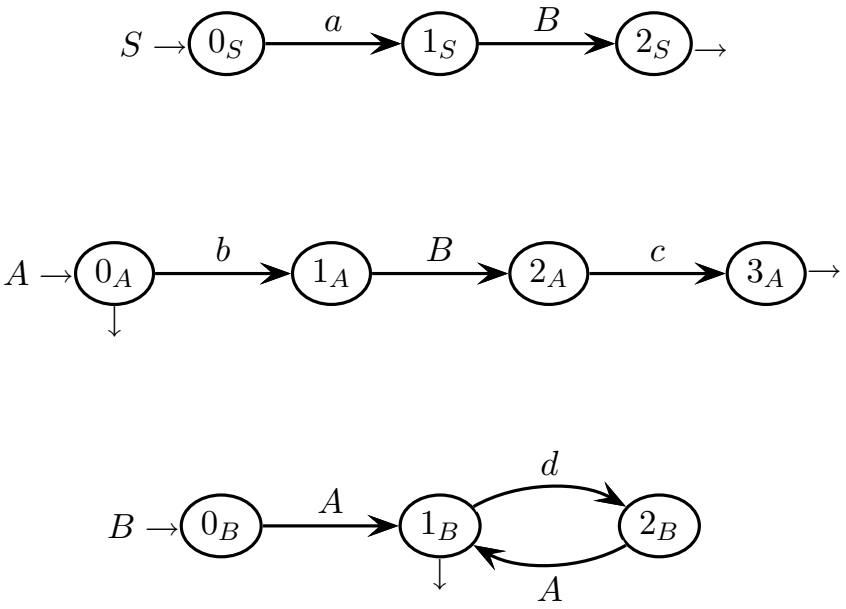
   not (**not** $\mathbf{n} =$ **n impl** $\mathbf{n} =$ **n**)

   Write a grammar that models the syntax described above. The grammar must be not ambiguous and should be in the extended form (*EBNF*).

---

3 Syntax Analysis and Parsing 20%

1. Consider the following grammar G, represented as a network of recursive machines (with axiom S):



Answer the following questions:

(a) Draw the complete pilot graph of grammar G, say if the grammar has the $ELR(1)$ property, and specify which m-states of the pilot have to be examined to prove the property.

(b) Say if grammar G has the $ELL(1)$ property, and decorate the *PCFG* graph of the grammar in the points where it is necessary to prove the property.

(c) (optional) Draw the *PCFG* graph of grammar G, and put all the call arcs and all the guide sets on it (the guide sets must decorate the call arcs and the exit darts of the final nodes).

---

4 Translation and Semantic Analysis 20%

1. Consider the following grammar G_s (with axiom A), which generates the Dyck language with one parenthesis type, namely the round brackets:

$$G_s \begin{cases} A & \rightarrow & \text{`(' } A \text{ `)' } B \mid \varepsilon \\ B & \rightarrow & \text{`(' } A \text{ `)' } A \mid \varepsilon \end{cases}$$

Answer the following questions:

(a) Write a syntactic transduction scheme G_τ (or a translation grammar), with grammar G_s as its source component. The scheme describes a translation τ such that, considering the series of the pairs of matching parentheses at the same depth level, the pairs placed at an odd position (first, third, etc.) are left unchanged, while those placed at an even position (second, fourth, etc.) are translated into square brackets. For instance:

$$\begin{aligned} \tau\big(\,(\,)\,(\,)\,(\,)\,\big) &= (\,)\,[\,]\,(\,) \\ \tau\Big(\,(\,)\,\big(\,(\,)\,(\,)\,\big)\,(\,)\,\Big) &= (\,)\,\big[\,(\,)\,[\,]\,\big]\,(\,) \end{aligned}$$

(b) (optional) Write a new syntactic transduction scheme $G_{\tau'}$ (or a translation grammar) that translates the round brackets into square ones in the same way as the previous scheme G_τ, but with the additional feature of putting a character "$\bullet$" (a point) into all the bracket pairs (round or square) that do not contain any more parentheses, i.e., into all the empty pairs. For instance:

$$\tau'\Big(\,(\,)\,\big(\,(\,)\,(\,)\,\big)\,(\,)\,\Big) = (\bullet)\,\big[\,(\bullet)\,[\bullet]\,\big]\,(\bullet)$$

A possible approach to write the new scheme $G_{\tau'}$, is the following (but he who prefers, is free to do differently, if he thinks it is better). Take a new source grammar G'_s, a variant of the old one G_s, that uses the following nonterminals:

- S, is the axiom
- A, which generates a parenthesis pair at the first position in the series of pairs at the same depth level
- B, which generates a parenthesis pair at an even position in the series of pairs at the same depth level
- C, which generates a parenthesis pair at an odd position (greater than one) in the series of pairs at the same depth level

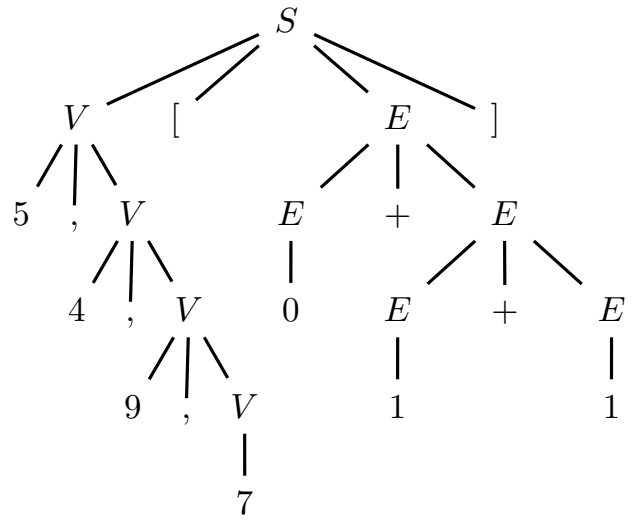
2. Take a vector V of numbers and a summation E of positive or null integers. Consider the problem of finding the following element of V:

```
V [i1 + i2 + ... + ik]
```

where $i1$, $i2$, ..., ik (with $k \geq 1$) are the integers of summation E. Suppose vector V is indexed as in the language C, starting from index 0.

The problem above is modeled by the following syntax (with axiom S), which generates the vector V (modeled as a list of numerical elements) and concatenates to V the summation E (modeled as an expression with addition):

$$\begin{cases} S & \rightarrow & V\ [\ E\ ] \\ V & \rightarrow & n\ ,\ V \\ V & \rightarrow & n \\ E & \rightarrow & E\ +\ E \\ E & \rightarrow & i \end{cases}$$



The terminals n and i schematize a generic element of vector V and a generic integer of summation E, respectively. Commas and brackets are just "syntactic sugar".

For instance suppose vector V has the four elements 5, 4, 9, 7. If one wishes one knew the value of $V[0+1+1]$, then the result is element 9 of V. The string that models this instance of the problem, is "$\underbrace{5,\ 4,\ 9,\ 7}_{vector}\ [\ \underbrace{0\ +\ 1\ +\ 1}_{summation}\ ]$"; see its tree above.

Answer the following questions:

(a) Write an attribute grammar that, as it is requested by the problem above, brings the wanted vector element to the tree root S. Define the necessary attributes and write the semantic functions coupled with the grammar rules (please use the tables already prepared on the next pages). In the semantic functions, use the terminal names n and i to mean the values of the terminals themselves.

(b) Say if the attribute grammar designed is of type one-sweep, and explain why. In the case the grammar is one-sweep, write the semantic evaluation procedure of axiom S, say if the grammar is of type L as well, and explain why.

(c) (optional) If the attribute grammar designed is of type one-sweep, write the semantic evaluation procedure of nonterminal V; otherwise compute a correct evaluation order of the attributes for the sample syntax tree shown above.

attributes to be used for the grammar

type	*name*	*domain*	*nonterm.*	*meaning*

#	*syntax*	*semantics* - to be completed
1:	$S_0 \rightarrow V_1 \; [\; E_2 \;]$	
2:	$V_0 \rightarrow n \; , \; V_1$	
3:	$V_0 \rightarrow n$	
4:	$E_0 \rightarrow E_1 \; + \; E_2$	
5:	$E_0 \rightarrow i$	